

Taller: Introducción a APIs REST con FastAPI

Gestión de tareas con persistencia en archivo JSON

Duración:	2 sesiones (4 horas)
Modalidad:	Práctica guiada
Requisitos:	Python 3.8+, pip, editor de texto
Entregable:	API funcional + archivo JSON

¿Usa Windows?

Si al seguir este taller tiene errores relacionados con `model_dump`, con anotaciones de tipo como `list[dict]`, o su instalación trajo Pydantic v1, consulte el **Apéndice A** al final de este documento.

1. Introducción

En este taller usted construirá, paso a paso, una API REST funcional utilizando FastAPI, un framework moderno de Python para el desarrollo de servicios web. A través de la implementación de un sistema sencillo de gestión de tareas (to-do list), aprenderá los conceptos fundamentales que necesitará para los proyectos posteriores del curso.

1.1. Objetivos de aprendizaje

- Comprender la arquitectura cliente-servidor y el estilo REST.
- Crear endpoints HTTP para las operaciones CRUD (Create, Read, Update, Delete).
- Utilizar modelos de datos con Pydantic para validación automática.
- Implementar persistencia básica mediante archivos JSON.
- Probar la API usando la documentación interactiva de Swagger UI.

1.2. ¿Por qué FastAPI?

FastAPI ofrece varias ventajas que lo hacen ideal para este taller:

- Documentación automática (**Swagger UI**): al ejecutar la aplicación, se genera automáticamente una interfaz en `/docs` donde se pueden probar todos los endpoints sin herramientas externas.
- Validación automática (**Pydantic**): los modelos de datos verifican tipos y restricciones sin necesidad de código manual.
- Sintaxis simple y concisa que facilita concentrarse en los conceptos.

1.3. Conceptos previos: REST en breve

REST (Representational State Transfer) es un estilo arquitectónico para diseñar servicios web. En REST, los recursos (en nuestro caso, tareas) se identifican mediante URLs y se manipulan a través de los métodos estándar de HTTP:

Método	Operación	Ejemplo de URL	Descripción
GET	Read	/tareas	Obtener todas las tareas
GET	Read	/tareas/{id}	Obtener una tarea específica
POST	Create	/tareas	Crear una nueva tarea
PUT	Update	/tareas/{id}	Actualizar una tarea existente
DELETE	Delete	/tareas/{id}	Eliminar una tarea

2. Sesión 1: Primeros pasos con FastAPI

2.1. Preparación del entorno

Abra una terminal y ejecute los siguientes comandos para crear el entorno de trabajo:

```
# Crear directorio del proyecto
mkdir taller-fastapi
cd taller-fastapi

# Crear y activar entorno virtual
python3 -m venv venv
source venv/bin/activate      # Linux/macOS
venv\Scripts\activate        # Windows

# Instalar dependencias
pip install fastapi uvicorn
```

¿Qué se instaló?

- fastapi: el framework para construir la API.
- uvicorn: un servidor ASGI que ejecutará la aplicación.

2.2. Hola mundo con FastAPI

Cree un archivo llamado main.py con el siguiente contenido:

```
# main.py
from fastapi import FastAPI

app = FastAPI(
    title="API de Gestión de Tareas",
    description="Taller de Sistemas Distribuidos - UIS",
    version="1.0.0"
)

@app.get("/")
def raiz():
    return {"mensaje": "Bienvenido a la API de Tareas"}
```

Ejecute la aplicación desde la terminal:

```
uvicorn main:app --reload
```

Abra su navegador en <http://127.0.0.1:8000> y deberá ver la respuesta JSON. Luego, visite <http://127.0.0.1:8000/docs> para acceder a la documentación interactiva de Swagger UI.

Nota sobre --reload

La opción `--reload` reinicia automáticamente el servidor cada vez que usted modifica el código. Es muy útil durante el desarrollo, pero no se debe usar en producción.

2.3. Definición del modelo de datos

FastAPI utiliza Pydantic para definir y validar la estructura de los datos. Cree un archivo llamado modelos.py:

```
# modelos.py
from pydantic import BaseModel
from typing import Optional

class TareaBase(BaseModel):
    titulo: str
    descripcion: Optional[str] = None
    completada: bool = False

class TareaCreate(TareaBase):
    """Modelo para crear una tarea (no incluye id)."""
    pass

class Tarea(TareaBase):
    """Modelo completo con id asignado por el servidor."""
    id: int
```

Observe que se definen dos modelos: `TareaCreate` (lo que envía el cliente, sin id) y `Tarea` (la respuesta completa, con id). Esta separación es una buena práctica porque el servidor es quien asigna el identificador.

2.4. Persistencia con archivo JSON

En lugar de una base de datos, se utilizará un archivo JSON para almacenar las tareas. Cree un archivo llamado `almacenamiento.py`:

```
# almacenamiento.py
import json
from pathlib import Path

ARCHIVO = Path("tareas.json")

def cargar_tareas() -> list[dict]:
    """Carga las tareas desde el archivo JSON."""
    if not ARCHIVO.exists():
        return []
    with open(ARCHIVO, "r", encoding="utf-8") as f:
        return json.load(f)

def guardar_tareas(tareas: list[dict]) -> None:
    """Guarda la lista de tareas en el archivo JSON."""
    with open(ARCHIVO, "w", encoding="utf-8") as f:
        json.dump(tareas, f, ensure_ascii=False, indent=2)

def siguiente_id(tareas: list[dict]) -> int:
    """Retorna el siguiente id disponible."""
    if not tareas:
        return 1
    return max(t["id"] for t in tareas) + 1
```

Si usa Windows y esto le genera un `TypeError: 'type' object is not subscriptable`, es un problema conocido de compatibilidad — revise la **Corrección 1 del Apéndice A**.

2.5. Endpoints de lectura y creación (GET y POST)

Actualice `main.py` para incorporar los modelos y el almacenamiento. Reemplace todo el contenido del archivo:

```
# main.py
from fastapi import FastAPI, HTTPException
from modelos import Tarea, TareaCreate
from almacenamiento import cargar_tareas, guardar_tareas, siguiente_id

app = FastAPI(
    title="API de Gestión de Tareas",
    description="Taller de Sistemas Distribuidos - UIS",
    version="1.0.0"
)

@app.get("/tareas", response_model=list[Tarea])
def listar_tareas():
    """Retorna todas las tareas almacenadas."""
    return cargar_tareas()

@app.get("/tareas/{tarea_id}", response_model=Tarea)
def obtener_tarea(tarea_id: int):
    """Retorna una tarea específica por su id."""
    tareas = cargar_tareas()
    for t in tareas:
        if t["id"] == tarea_id:
            return t
    raise HTTPException(status_code=404, detail="Tarea no encontrada")

@app.post("/tareas", response_model=Tarea, status_code=201)
def crear_tarea(tarea: TareaCreate):
    """Crea una nueva tarea y la almacena."""
    tareas = cargar_tareas()
    nueva = tarea.model_dump()
    nueva["id"] = siguiente_id(tareas)
    tareas.append(nueva)
    guardar_tareas(tareas)
    return nueva
```

Si al crear una tarea obtiene un error HTTP 500, o ve subrayado `list[Tarea]` o `model_dump`, su instalación probablemente trae Pydantic v1 o Python 3.8 — vea las **Correcciones 2 y 3 del Apéndice A**.

Reinicie el servidor (o espere a que `--reload` lo haga automáticamente) y vaya a `http://127.0.0.1:8000/docs`. Desde allí podrá probar la creación y consulta de tareas directamente desde el navegador.

Ejercicio 1 — Sesión 1

1. Cree al menos 3 tareas usando la interfaz de Swagger (POST `/tareas`).
2. Verifique que las tareas se crearon correctamente (GET `/tareas`).
3. Consulte una tarea individual por su id (GET `/tareas/{tarea_id}`).
4. Intente consultar una tarea con un id que no exista y observe la respuesta 404.
5. Abra el archivo `tareas.json` con un editor de texto y verifique su contenido.

3. Sesión 2: CRUD completo y mejoras

3.1. Endpoints de actualización y eliminación (PUT y DELETE)

Agregue los siguientes endpoints al final de main.py:

```
@app.put("/tareas/{tarea_id}", response_model=Tarea)
def actualizar_tarea(tarea_id: int, datos: TareaCreate):
    """Actualiza todos los campos de una tarea existente."""
    tareas = cargar_tareas()
    for i, t in enumerate(tareas):
        if t["id"] == tarea_id:
            tareas[i] = {**datos.model_dump(), "id": tarea_id}
            guardar_tareas(tareas)
            return tareas[i]
    raise HTTPException(status_code=404, detail="Tarea no encontrada")

@app.delete("/tareas/{tarea_id}", status_code=204)
def eliminar_tarea(tarea_id: int):
    """Elimina una tarea por su id."""
    tareas = cargar_tareas()
    tareas_filtradas = [t for t in tareas if t["id"] != tarea_id]
    if len(tareas_filtradas) == len(tareas):
        raise HTTPException(status_code=404, detail="Tarea no encontrada")
    guardar_tareas(tareas_filtradas)
```

El `model_dump` de `actualizar_tarea` tiene el mismo problema de compatibilidad que el de la sección 2.5: use la **Corrección 2 del Apéndice A** si aplica.

3.2. Filtrado con parámetros de consulta

Una funcionalidad útil es poder filtrar las tareas por su estado (completadas o pendientes). Modifique el endpoint de listado en main.py:

```
from typing import Optional

@app.get("/tareas", response_model=list[Tarea])
def listar_tareas(completada: Optional[bool] = None):
    """
    Retorna las tareas almacenadas.
    Si se proporciona el parámetro completada, filtra por estado.
    Ejemplo: GET /tareas?completada=true
    """
    tareas = cargar_tareas()
    if completada is not None:
        tareas = [t for t in tareas if t["completada"] == completada]
    return tareas
```

Ahora puede usar `GET /tareas?completada=true` para ver solo las tareas terminadas, o `GET /tareas?completada=false` para las pendientes.

3.3. Pruebas con curl (opcional)

Además de Swagger UI, puede probar la API desde la terminal usando curl. Esto es especialmente útil en contextos de automatización y scripts:

```
# Crear una tarea
curl -X POST http://127.0.0.1:8000/tareas \
```

```

-H "Content-Type: application/json" \
-d '{"titulo": "Estudiar REST", "descripcion": "Leer capítulo 4"}'

# Listar todas las tareas
curl http://127.0.0.1:8000/tareas

# Obtener tarea con id 1
curl http://127.0.0.1:8000/tareas/1

# Actualizar tarea con id 1
curl -X PUT http://127.0.0.1:8000/tareas/1 \
-H "Content-Type: application/json" \
-d '{"titulo": "Estudiar REST", "completada": true}'

# Eliminar tarea con id 1
curl -X DELETE http://127.0.0.1:8000/tareas/1

```

Ejercicio 2 — Sesión 2

1. Cree al menos 5 tareas con distintos estados.
2. Actualice el título y estado de al menos 2 tareas usando PUT.
3. Elimine una tarea y verifique que ya no aparece en el listado.
4. Use el parámetro de consulta para filtrar las tareas completadas.
5. Pruebe al menos 2 operaciones usando curl desde la terminal.

3.4. Estructura final del proyecto

Al finalizar ambas sesiones, el proyecto deberá tener la siguiente estructura de archivos:

```

taller-fastapi/
├── venv/           # Entorno virtual (no se entrega)
├── main.py         # Aplicación principal con los endpoints
├── modelos.py      # Modelos de datos (Pydantic)
├── almacenamiento.py # Funciones de lectura/escritura JSON
└── tareas.json     # Datos persistidos (se genera automáticamente)

```

4. Entregables y criterios de evaluación

Cada estudiante (o grupo, según lo indicado por el profesor) debe entregar:

1. El código fuente completo del proyecto (sin la carpeta venv).
2. El archivo tareas.json con al menos 5 tareas de prueba.
3. Un breve documento (máximo 1 página) donde explique:
 - ¿Qué es una API REST y cuáles son sus métodos principales?
 - ¿Qué ventaja tiene usar modelos Pydantic para validación?
 - ¿Qué limitaciones tiene el almacenamiento en archivo JSON frente a una base de datos?

5. Desafíos adicionales (opcionales)

Para los estudiantes que deseen profundizar, se proponen los siguientes retos adicionales. Estos no son obligatorios, pero demuestran un nivel de comprensión superior:

1. PATCH parcial: implemente un endpoint PATCH /tareas/{id} que permita actualizar solo algunos campos de la tarea (a diferencia de PUT, que reemplaza todos los campos).
2. Marca de tiempo: agregue un campo fecha_creacion que se asigne automáticamente al crear cada tarea (investigue datetime en Python).
3. Búsqueda por texto: implemente un parámetro de consulta buscar que filtre las tareas cuyo título o descripción contenga una cadena de texto dada.
4. Cliente HTTP: escriba un pequeño script en Python que use la librería requests para interactuar con la API programáticamente (crear, listar y eliminar tareas).

Recursos útiles

- Documentación oficial de FastAPI: <https://fastapi.tiangolo.com/es/>
- Documentación de Pydantic: <https://docs.pydantic.dev/>
- Tutorial interactivo de HTTP: <https://developer.mozilla.org/es/docs/Web/HTTP>
- Códigos de estado HTTP: <https://developer.mozilla.org/es/docs/Web/HTTP/Status>

Apéndice A · Fe de erratas

Compatibilidad en Windows con Python 3.8 y Pydantic v1

El código de este taller utiliza sintaxis de Python 3.9+ y Pydantic v2. Si usted tiene una versión anterior de Python (3.8) o si su instalación de FastAPI trajo Pydantic v1 (frecuente en algunas instalaciones de Windows), es necesario aplicar las siguientes correcciones. Para verificar sus versiones:

```
python3 --version
pip show pydantic | grep Version
```

Alternativa rápida

Si su versión de Pydantic es 2.x y Python es 3.9+, el código original funciona sin cambios. Solo aplique estas correcciones si experimenta errores.

Corrección 1 — almacenamiento.py (anotaciones de tipo)

Las anotaciones `list[dict]` requieren Python 3.9+. Para compatibilidad con Python 3.8, se debe importar `List` desde `typing`.

✗ Donde dice:

```
ARCHIVO = Path("tareas.json")

def cargar_tareas() -> list[dict]:
```

✓ Debe decir:

```
from typing import List

ARCHIVO = Path("tareas.json")

def cargar_tareas() -> List[dict]:
```

Aplique el mismo cambio en `guardar_tareas(tareas: List[dict])` y `siguiente_id(tareas: List[dict])`.

Corrección 2 — main.py (model_dump → dict)

El método `model_dump()` fue introducido en Pydantic v2. En Pydantic v1, el equivalente es `dict()`. Este es el error que causa el HTTP 500 al crear tareas.

✗ Donde dice (en crear_tarea y actualizar_tarea):

```
nueva = tarea.model_dump()
# y en actualizar_tarea:
tareas[i] = {**datos.model_dump(), "id": tarea_id}
```

✓ Debe decir:

```
nueva = tarea.dict()
# y en actualizar_tarea:
tareas[i] = {**datos.dict(), "id": tarea_id}
```

Corrección 3 — main.py (response_model con List)

De forma similar, `list[Tarea]` en los decoradores requiere Python 3.9+. Use `List[Tarea]` para compatibilidad.

✗ Donde dice:

```
from modelos import Tarea, TareaCreate

@app.get("/tareas", response_model=list[Tarea])
```

✓ Debe decir:

```
from typing import List, Optional
from modelos import Tarea, TareaCreate

@app.get("/tareas", response_model=List[Tarea])
```

Aplique el mismo cambio en el decorador de la sección 3.2 (listado con filtro por estado).

Consejo

Para evitar estos problemas de compatibilidad, se recomienda instalar versiones recientes con: `pip install --upgrade fastapi uvicorn pydantic`